Kubernetes

A Book-Style Learning Outline

This document turns the current Kubernetes notes into a book-style roadmap. It is designed for someone who wants to go from Docker familiarity to real Kubernetes fluency.

The goal is not only to memorize objects such as Pods and Services, but to build the mental model needed to deploy, debug, and operate applications in a cluster.

Title Page

Title: Kubernetes
Subtitle: From First Cluster to Production Thinking
Audience: Backend engineers, DevOps learners, platform engineers, and interview candidates
Prerequisites: Docker, containers, basic Linux, networking fundamentals, YAML basics

---

Preface

Kubernetes is easier to learn when studied in layers.

Most people get stuck because they learn objects in isolation:

- Pod
- Deployment
- Service
- ConfigMap
- Secret

That approach creates vocabulary, but not understanding.

This outline is organized like a book:

1. Build the mental model.
2. Learn core workload and networking objects.
3. Deploy real applications.
4. Add production concerns.
5. Study internals and platform-level topics.

How To Use This Book

Read the parts in order.

Each chapter should answer three questions:

1. What problem does this Kubernetes feature solve?
2. How does it fit into the system?
3. What should you practice with `kubectl` right now?

Recommended workflow for each chapter:

1. Read the concept.
2. Apply one small manifest.
3. Inspect resources with `kubectl get` and `kubectl describe`.
4. Break something on purpose.
5. Observe how Kubernetes responds.

Table of Contents

Part I. Foundations

Part II. Core Workloads

5. Pods
6. ReplicaSets
7. Deployments
8. Namespaces
9. Labels, Selectors, and Annotations

Part III. Networking and Traffic

10. Services
11. DNS and Service Discovery
12. Ingress
13. Network Policies

Part IV. Configuration and Persistence

14. ConfigMaps
15. Secrets
16. Volumes
17. Persistent Volumes and Persistent Volume Claims
18. StatefulSets

Part V. Running Real Applications

19. Multi-Tier Application Design
20. Health Checks with Probes
21. Resource Requests and Limits
22. Jobs and CronJobs
23. Autoscaling

Part VI. Production Operations

24. Rolling Updates and Rollbacks
25. Observability: Logs, Metrics, and Events

Part VII. Cluster Internals and Advanced Topics

Appendices

Part I. Foundations

Chapter 1. Why Kubernetes Exists

Purpose: Understand the gap between containers and orchestration.

Topics:

- Containers vs container orchestration
- Docker Compose vs Kubernetes
- Self-healing
- Scaling
- Service discovery
- Rolling updates

- Declarative infrastructure

Core idea:
Docker runs containers. Kubernetes keeps systems in the desired state.

Hands-on outcomes:

- Create a local cluster with `kind` or `minikube`
- Run the first Deployment
- Inspect Pods and nodes

Chapter 2. Declarative Thinking and Desired State

Purpose: Learn the main mental model behind Kubernetes.

Topics:

- Imperative vs declarative operations
- Desired state
- Reconciliation loop
- Controllers
- Why manual fixes do not last

Key sentence:
In Kubernetes, you describe what should exist, and controllers work continuously to make reality match.

Chapter 3. Cluster Architecture

Purpose: Understand the big picture before learning individual resources.

Topics:

- Cluster
- Control plane
- Worker nodes
- API server
- etcd

- Scheduler
- Controller manager
- kubelet
- container runtime
- kube-proxy

Diagram:

```
kubectl -> API Server -> etcd
                     -> Scheduler
                     -> Controllers

Worker Node -> kubelet
            -> container runtime
            -> kube-proxy
```

Chapter 4. The Kubernetes API and Object Model

Purpose: See Kubernetes as an API platform, not only a CLI tool.

Topics:

- API resources
- `apiVersion`, `kind`, `metadata`, `spec`, `status`
- Namespaced vs cluster-scoped resources
- `kubectl apply`
- Declarative manifests

Important point:
Nearly everything in Kubernetes is an API object.

Part II. Core Workloads

Chapter 5. Pods

Purpose: Learn the smallest deployable unit.

Topics:

- What a Pod is
- One-container Pods
- Multi-container Pods
- Shared network namespace
- Shared volumes
- Pod lifecycle
- Why Pods are ephemeral

Practice:

- Create a single Pod
- Read logs
- Exec into the container
- Delete the Pod and observe behavior

Chapter 6. ReplicaSets

Purpose: Understand how Kubernetes keeps Pods alive.

Topics:

- Replica management
- Desired replica count
- Selector matching
- Why ReplicaSets exist
- Why they are usually managed by Deployments

Practice:

- Create a ReplicaSet with 3 replicas
- Delete one Pod
- Watch Kubernetes recreate it

Chapter 7. Deployments

Purpose: Learn the standard way to run stateless applications.

Topics:

- Deployment -> ReplicaSet -> Pod hierarchy
- Rolling updates
- Rollbacks
- Version history
- Scaling

Practice:

- Deploy `nginx`
- Scale up and down
- Change image version
- Roll back a failed rollout

Chapter 8. Namespaces

Purpose: Organize and isolate cluster resources.

Topics:

- Logical separation
- Environment partitioning
- Default namespace behavior
- Resource scoping
- Quotas and policies by namespace

Practice:

- Create `dev` and `prod`
- Deploy the same app in both
- Compare results with namespace filtering

Chapter 9. Labels, Selectors, and Annotations

Purpose: Learn the metadata model that connects Kubernetes resources.

Topics:

- Labels for grouping and selection

- Selectors in Services, ReplicaSets, and Deployments
- Annotation use cases
- Common mismatch problems

Important point:
Many Kubernetes failures are really label and selector mismatches.

Part III. Networking and Traffic

Chapter 10. Services

Purpose: Understand stable networking for ephemeral Pods.

Topics:

- Why Pod IPs are not enough
- ClusterIP
- NodePort
- LoadBalancer
- Endpoints
- Port vs targetPort vs nodePort

Practice:

- Expose a Deployment
- Inspect `kubectl get svc`
- Check endpoints
- Test traffic flow from another Pod

Chapter 11. DNS and Service Discovery

Purpose: Learn how workloads find each other.

Topics:

- Cluster DNS
- Service names

- Namespace-qualified names
- `service.namespace.svc.cluster.local`
- DNS troubleshooting

Practice:

- Launch a temporary debug Pod
- Resolve a Service name
- Call it from inside the cluster

Chapter 12. Ingress

Purpose: Route HTTP and HTTPS traffic into the cluster.

Topics:

- Ingress vs Service exposure
- Ingress controllers
- Host-based routing
- Path-based routing
- TLS termination

Practice:

- Route `/api` and `/app` to different Services
- Add a local host rule

Chapter 13. Network Policies

Purpose: Control Pod-to-Pod communication.

Topics:

- Default allow model
- Namespace and Pod selectors
- Ingress rules
- Egress rules
- Zero-trust cluster networking basics

Production value:
Without NetworkPolicy, clusters are often more open than teams expect.

Part IV. Configuration and Persistence

Chapter 14. ConfigMaps

Purpose: Store non-sensitive configuration outside the image.

Topics:

- Environment-specific values
- Environment variables
- Mounted files
- Reload considerations

Practice:

- Inject app config through a ConfigMap
- Mount config as a file

Chapter 15. Secrets

Purpose: Handle sensitive data more safely.

Topics:

- Secret types
- Base64 encoding vs encryption
- Environment variable injection
- Volume mounts
- Secret rotation concerns

Important note:
Secrets are not automatically secure just because they are called Secrets. Storage and access control still matter.

Chapter 16. Volumes

Purpose: Separate runtime data from container filesystem assumptions.

Topics:

- EmptyDir
- ConfigMap and Secret volumes
- Shared storage inside a Pod
- Ephemeral vs persistent data

Chapter 17. Persistent Volumes and Persistent Volume Claims

Purpose: Give stateful workloads durable storage.

Topics:

- PV and PVC model
- Storage classes
- Dynamic provisioning
- Access modes
- Reclaim policies

Practice:

- Create a PVC
- Mount it into a Pod
- Delete and recreate the Pod
- Verify the data persists

Chapter 18. StatefulSets

Purpose: Run workloads that need stable identity and stable storage.

Topics:

- Ordered identity
- Stable Pod naming

- Headless Services
- Persistent storage per replica
- Database and queue use cases

Part V. Running Real Applications

Chapter 19. Multi-Tier Application Design

Purpose: Translate a Docker Compose app into Kubernetes resources.

Topics:

- Frontend, backend, database separation
- Internal and external traffic flow
- Config and secrets per service
- Storage decisions

Practice project:

- Convert a small Node.js + database app into Kubernetes manifests

Chapter 20. Health Checks with Probes

Purpose: Teach Kubernetes how to judge application health.

Topics:

- Liveness probes
- Readiness probes
- Startup probes
- Common probe mistakes
- Impact on rollouts and traffic

Practice:

- Add probes to an API
- Simulate unhealthy states
- Observe restart and readiness behavior

Chapter 21. Resource Requests and Limits

Purpose: Make scheduling and fairness predictable.

Topics:

- CPU and memory requests
- CPU and memory limits
- Scheduling implications
- OOMKilled behavior
- Namespace quotas
- LimitRanges

Repo connection:
The `infra/4resource-quotas/` examples in this repository are a good fit for this chapter.

Chapter 22. Jobs and CronJobs

Purpose: Run finite and scheduled workloads.

Topics:

- Job lifecycle
- Completion tracking
- Retries and backoff
- Cron syntax in CronJobs
- Cleanup strategy

Use cases:

- Migrations
- Reports
- Backups
- Scheduled sync tasks

Chapter 23. Autoscaling

Purpose: Scale workloads based on demand.

Topics:

- Horizontal Pod Autoscaler
- Metrics requirements
- CPU and custom metrics
- Scaling behavior and caveats
- Vertical Pod Autoscaler overview

Part VI. Production Operations

Chapter 24. Rolling Updates and Rollbacks

Purpose: Release changes safely.

Topics:

- Deployment strategies
- Max unavailable
- Max surge
- Rollback triggers
- Observing rollout status

Chapter 25. Observability: Logs, Metrics, and Events

Purpose: Make the cluster explain itself.

Topics:

- `kubectl logs`
- `kubectl describe`
- events
- Prometheus
- Grafana
- centralized logging
- tracing overview

Debugging rule:
Before changing manifests, inspect events and object descriptions.

Chapter 26. RBAC and Service Accounts

Purpose: Control who can do what.

Topics:

- Authentication vs authorization
- Roles and ClusterRoles
- RoleBindings and ClusterRoleBindings
- Service accounts
- Least privilege

Chapter 27. Scheduling, Affinity, Taints, and Tolerations

Purpose: Influence where workloads run.

Topics:

- Node selectors
- Node affinity
- Pod affinity
- Pod anti-affinity
- Taints
- Tolerations
- Dedicated nodes

Chapter 28. Helm and Kustomize

Purpose: Manage reusable and environment-specific configuration.

Topics:

- Helm charts
- values files

- release management
- Kustomize bases and overlays
- when to use each tool

Chapter 29. GitOps

Purpose: Operate clusters through version-controlled desired state.

Topics:

- Pull-based deployment
- Argo CD
- Flux
- Drift detection
- Auditability
- Promotion across environments

Part VII. Cluster Internals and Advanced Topics

Chapter 30. etcd, Scheduler, Controller Manager, and kubelet

Purpose: Deepen architectural understanding.

Topics:

- How state is stored
- How scheduling decisions are made
- How controllers reconcile
- How kubelet enforces Pod execution

Chapter 31. CNI, CSI, and kube-proxy

Purpose: Understand how networking and storage are implemented.

Topics:

- CNI overview
- Popular CNIs such as Calico and Cilium
- CSI overview
- kube-proxy modes
- Service routing internals

Chapter 32. CRDs and Operators

Purpose: Learn how Kubernetes becomes an extensible platform.

Topics:

- Custom Resource Definitions
- Custom controllers
- Operator pattern
- Real examples from databases and platforms

Chapter 33. Security Hardening

Purpose: Move from functional clusters to safer clusters.

Topics:

- Pod security standards
- image trust
- secret handling
- RBAC hygiene
- network isolation
- admission controls
- runtime restrictions

Chapter 34. Upgrades, Backups, and Disaster Recovery

Purpose: Operate Kubernetes as infrastructure.

Topics:

- Control plane upgrades
- Worker node upgrades
- draining nodes
- etcd backup strategy
- restore planning
- failure testing

Appendix A. Essential `kubectl` Commands

Cluster and Nodes

```
kubectl cluster-info
kubectl get nodes
kubectl describe node <node-name>
```

Workloads

```
kubectl get pods
kubectl get deploy
kubectl get rs
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- sh
```

Configuration and Networking

```
kubectl get svc
kubectl get endpoints
kubectl get configmap
kubectl get secret
kubectl describe svc <service-name>
```

Rollouts

```
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>
kubectl scale deployment/<name> --replicas=3
```

Namespace-Aware Usage

```
kubectl get pods -n dev
kubectl get all -n prod
kubectl config set-context --current --namespace=dev
```

Appendix B. Common Debugging Playbooks

Pod Will Not Start

Check:

- `kubectl describe pod <name>`
- events
- image name
- pull failures
- resource limits
- probe failures

Service Has No Traffic

Check:

- selector match
- target port

- endpoints
- Pod readiness
- namespace mismatch

Deployment Stuck During Rollout

Check:

- readiness probe
- image version
- scheduling constraints
- insufficient CPU or memory
- failing application startup

PVC Will Not Bind

Check:

- storage class
- requested size
- access mode
- available provisioner

Appendix C. Study Plan by Month

Month 1

- Parts I and II
- Build a local cluster
- Practice Pods, ReplicaSets, Deployments, and Namespaces

Month 2

- Part III
- Practice Services, DNS, and Ingress

- Debug label and selector issues

Month 3

- Part IV
- Practice ConfigMaps, Secrets, PVCs, and StatefulSets

Month 4

- Part V
- Deploy a real application
- Add probes, quotas, and autoscaling

Month 5

- Part VI
- Learn Helm, RBAC, and observability
- Introduce GitOps thinking

Month 6 and Beyond

- Part VII
- Study internals, CRDs, Operators, and disaster recovery

Appendix D. Repo-Based Practice Labs

This repository already contains useful material in `infra/`.

Lab 1. Labels and Selectors

Use:

- `infra/3labels-selectors/color-api.yml`
- `infra/3labels-selectors/color-depl.yml`

Practice:

- inspect labels
- query with `-l`
- compare Deployment selectors with Pod labels

Lab 2. Namespaces and Resource Quotas

Use:

- `infra/4resource-quotas/dev-ns.yml`
- `infra/4resource-quotas/prod-ns.yml`
- `infra/4resource-quotas/color-api-depl.yml`
- `infra/4resource-quotas/color-api.-pod.yml`

Practice:

- create namespaces
- apply workloads into `dev`
- inspect quota impact
- observe why a Deployment may show `0/N` even when similarly labeled Pods exist

Lab 3. Deployment vs Standalone Pod

Goal:

- understand that a matching label does not mean a Pod is owned by a Deployment
- compare controller-managed Pods with manually created Pods

Closing Note

If you study Kubernetes in this order, the platform stops feeling like a huge list of YAML resources and starts feeling like a coherent operating model:

- declare intent
- let controllers reconcile state
- expose stable interfaces

- separate config from code
- design for failure
- operate through repeatable workflows

That is the shift from learning Kubernetes terminology to thinking like a Kubernetes engineer.